

GBCR Platform Reference Documentation

Date: 2026-03-30

Project Path: `/Users/eliyazar/Documents/MX Project/gbcr-platform`

Prepared For: project reference and handover use

1. Executive Summary

GBCR Platform is a Next.js web application for Goldbell Car Rental operations. The current codebase combines operational fleet visibility, booking management, inspection workflows, service and maintenance views, customer lookup, analytics, user management, and AI-assisted insights inside a single authenticated dashboard.

The system integrates multiple back-end systems:

- `GBCR\_Platform` for application-owned data such as users, bookings, vehicle overrides, inspections, damage records, photos, and audit logs.
- `MAXDB76` for live operational fleet and work-order data from Maximo.
- AX / D365 Finance & Operations data for analytics and agreement invoicing views.
- Local or network file storage for inspection images and signatures.
- Microsoft Entra ID for SSO.
- OpenAI for streaming AI chat and data-assisted operational insight features.

The repository also includes prior design and planning documents for foundation architecture, vehicle fleet enhancements, a planned mobile app, and a mobile-responsive/PWA version. The current implementation aligns most strongly with the foundation, vehicle fleet, and mobile-responsive design work.

2. What The System Does Today

The implemented product areas visible in the repository are:

- Authentication with password login and Microsoft sign-in.
- Protected dashboard shell with desktop and mobile navigation.
- Fleet list view with search, status/category filters, and pagination.
- Vehicle detail view with editable rental overrides stored in the platform database.
- Booking management with create, status transitions, cancellation, deletion, and calendar view.
- Customer lookup with paging and active-rental filtering.
- Inspection management, including creation, checklist data, damage marking, photo upload, signatures, and submit workflow.
- Service and maintenance browsing from Maximo work orders.
- Analytics page backed by AX / D365 invoice and agreement data plus Maximo work-order data.
- AI insights page with chat, forecast, maintenance scoring, and anomaly feeds.

- User management for super admins.
- PWA basics including manifest, service worker registration, and offline fallback.
- Socket.io server for real-time event propagation.

3. Technology Stack

3.1 Front End

- Next.js `16.1.6`
- React `19.2.3`
- TypeScript `5`
- Tailwind CSS `4`
- Recharts for charts
- TanStack React Table for tabular grid behavior
- SWR present in dependencies, though current pages mostly use direct `fetch`

3.2 Back End

- Next.js Route Handlers under `src/app/api`
- Node.js Socket.io sidecar server in `server.js`
- `mssql` for SQL Server connectivity
- `jose` for JWT creation and verification
- `bcryptjs` for password hashing
- `uuid` for booking IDs
- `openai` SDK for AI chat streaming

3.3 Operational Characteristics

- Client-heavy dashboard pages (`use client` is common across main views)
- SQL queries embedded directly in route handlers
- Multiple SQL connection helpers for different systems
- File-based inspection asset storage
- Cookie-based auth with JWT
- Strong dependency on live enterprise data sources

4. High-Level Architecture

4.1 Application Layers

1. User accesses Next.js application.
2. Middleware checks the `gbcr\_token` cookie for protected routes.

3. Dashboard layout fetches `/api/auth/me` to resolve current user context.
4. Page-level React components fetch domain-specific APIs.
5. Route handlers query one of several SQL back ends or file storage helpers.
6. Responses return JSON for UI rendering, uploads, and workflow transitions.

4.2 Main Runtime Boundaries

- `src/app`: page routes and API routes.
- `src/components`: UI modules, layout shell, dashboards, AI components, inspection widgets.
- `src/lib`: auth, DB connectors, file storage, navigation, shared types, audit logging, utility formatting.
- `public`: manifest, service worker, offline page, icons, static brand assets.
- `storage`: default local file storage root for inspection files.
- `docs/superpowers`: prior design specs and implementation plans.

4.3 Data Source Responsibilities

- `db.ts`: app-owned read/write connection to `GBCR_Platform`, used for auth, users, inspections, and audit logs.
- `platformdb.ts`: connection to `GBCR_Platform`, used primarily for bookings and booking-oriented reads.
- `maxdb.ts`: read-only operational access to Maximo `MAXDB76`.
- `axdb.ts`: AX / D365 F&O connection, used for analytics invoice and agreement data.

5. Repository Structure

5.1 Important Root Files

- `package.json`: scripts, dependencies, and project identity.
- `next.config.ts`: Next.js configuration.
- `server.js`: Socket.io event server.
- `.env.example`: expected environment variables.
- `README.md`: generic Next.js template, not project-specific.

5.2 Important Source Directories

- `src/app/(auth)`: login and password change routes.
- `src/app/(dashboard)`: all authenticated pages and dashboard layout.
- `src/app/api`: application API surface.
- `src/components/layout`: shell, headers, nav, service worker registration.
- `src/components/dashboard`: dashboard charts.
- `src/components/inspection`: mobile inspection workflow pieces.
- `src/components/ui`: reusable presentational controls.

- ``src/lib``: auth, DB, audit, utilities, file storage.
- ``src/types``: auth and inspection domain types.

6. Authentication, Authorization, And Session Model

6.1 Authentication Modes

The platform supports:

- Password login via ``/api/auth/login``
- Microsoft Entra ID SSO via ``/api/auth/sso`` and ``/api/auth/sso/callback``

6.2 Session Model

- JWT cookie name: ``gbcr_token``
- Cookie max age: 24 hours
- Cookie is ``httpOnly``
- ``sameSite`` is ``lax`` for token set during sign-in
- Secure cookie is enabled only when ``NODE_ENV=production`` and ``USE_HTTPS=true``

6.3 Middleware Behavior

``src/middleware.ts``:

- Allows public access to login, auth SSO endpoints, static assets, manifest, favicon, service worker, and offline page.
- Returns ``401`` JSON for unauthenticated ``/api/*`` calls.
- Redirects unauthenticated page requests to ``/login?redirect=<path>``.
- Allows ``/change-password`` when authenticated.

6.4 Roles In The Codebase

Declared roles:

- ``super_admin``
- ``branch_manager``
- ``customer_service``
- ``rental_officer``
- ``inspector``
- ``driver``
- ``finance``

6.5 Navigation By Role

The navigation map in ``src/lib/nav-items.ts`` exposes modules selectively:

- Dashboard: all users

- Fleet: super admin, branch manager, customer service, rental officer
- Bookings: super admin, branch manager, customer service, rental officer
- Inspections: super admin, branch manager, rental officer, inspector
- Services: super admin, branch manager, rental officer, inspector
- Customers: super admin, branch manager, customer service, rental officer
- Analytics: super admin, branch manager, finance
- AI Insights: super admin, branch manager
- Settings: super admin only

6.6 Audit Logging

Most auth and write actions call `logAudit(...)`, which inserts into `audit_logs` with:

- acting user
- action name
- entity type and optional entity ID
- old and new values as JSON
- IP address if present

7. Dashboard Shell And User Experience

7.1 Root Layout

`src/app/layout.tsx`:

- sets metadata and viewport
- references the PWA manifest
- registers app icons
- mounts `ServiceWorkerRegistration`

7.2 Dashboard Layout

`src/app/(dashboard)/layout.tsx`:

- fetches `/api/auth/me` on mount
- redirects unauthenticated users to `/login`
- redirects users with `mustChangePassword` to `/change-password`
- renders desktop sidebar and header
- renders separate mobile header and bottom navigation

7.3 Visual System

The current UI style is a polished industrial dashboard aesthetic:

- charcoal primary surfaces
- Goldbell-aligned gold accent color

- custom cards, badges, buttons, skeletons, responsive tables
- strong mobile accommodations for lists, forms, and modal presentation

8. Main Pages

8.1 Dashboard Home

File: ``src/app/(dashboard)/page.tsx``

Purpose:

- landing summary for logged-in users
- top-level operational KPI snapshot

Data fetched:

- ``/api/fleet/stats``
- ``/api/auth/me``

Main UI elements:

- personalized greeting banner
- six KPI cards
- two fleet charts
- quick links to fleet, bookings, and user management

8.2 Fleet Management

Files:

- ``src/app/(dashboard)/fleet/page.tsx``
- ``src/app/(dashboard)/fleet/[assetnum]/page.tsx``

List view capabilities:

- search by asset, registration number, description, or model
- status filter
- category filter
- server-driven pagination
- sortable desktop table
- mobile card presentation

Vehicle detail capabilities:

- read-only vehicle, insurance, and assignment information from Maximo
- editable platform-owned fields:
 - category
 - availability override
 - override reason
 - notes

8.3 Bookings

Files:

- ``src/app/(dashboard)/bookings/page.tsx``
- ``src/app/(dashboard)/bookings/calendar/page.tsx``

Capabilities:

- list bookings with status filter
- create new booking
- status transitions:
 - ``PENDING -> CONFIRMED``
 - ``CONFIRMED -> ACTIVE``
 - ``ACTIVE -> COMPLETED``
- cancel booking
- permanently delete booking
- calendar month view based on overlapping date range logic
- mobile FAB for creation

8.4 Customers

File: ``src/app/(dashboard)/customers/page.tsx``

Capabilities:

- searchable customer list
- filter for customers with active rentals only
- paging and sorting
- mobile card rendering

Note:

The page attempts to navigate to ``/customers/<customer_code>`` on click, but a corresponding customer detail page was not observed in the current route set.

8.5 Inspections

File: ``src/app/(dashboard)/inspections/page.tsx``

Capabilities:

- list and filter inspections by type, status, and vehicle
- create new inspection
- navigate to inspection detail page
- mobile-friendly table/card behavior

Inspection subsystem capabilities across the repo:

- checklist and notes persistence
- damage diagram with positioned pins

- photo capture / upload
- inspector and customer signatures
- submit validation requiring key fields and at least one photo

8.6 Services And Maintenance

Files:

- ``src/app/(dashboard)/services/page.tsx``
- ``src/app/(dashboard)/services/[wonum]/page.tsx``

Capabilities:

- browse Maximo work orders
- filter by status and work type
- search by work order, asset, or description
- inspect labor and material line items
- cost summary per work order

8.7 Analytics

File: ``src/app/(dashboard)/analytics/page.tsx``

Purpose:

- management reporting view blending D365 and Maximo data

Displayed information:

- active agreement counts
- invoice totals
- 12-month revenue
- work-order volume
- chart-based trend views
- top customers
- top active sales orders
- agreement status breakdown

8.8 AI Insights

File: ``src/app/(dashboard)/ai/page.tsx``

Tabs:

- Chat
- Demand Forecast
- Maintenance
- Anomalies

Data sources:

- OpenAI API
- Maximo fleet and work-order data

8.9 Settings And User Management

Files:

- ``src/app/(dashboard)/settings/page.tsx``
- ``src/app/(dashboard)/settings/users/page.tsx``

Capabilities:

- currently exposes user management as the main settings area
- super-admin-only user listing and filtering
- create user
- edit user
- reset password
- deactivate user

8.10 Authentication Screens

Files:

- ``src/app/(auth)/login/page.tsx``
- ``src/app/(auth)/change-password/page.tsx``

Capabilities:

- Microsoft sign-in primary path
- collapsible password login fallback
- password change enforcement flow

9. Inspection Workflow In Detail

The inspection subsystem is one of the most fully defined operational workflows in the codebase.

9.1 Workflow States

Inspection statuses referenced in code:

- ``draft``
- ``in_progress``
- ``submitted``
- ``reviewed``
- ``approved``
- ``disputed``
- ``void``

9.2 Typical Flow

1. User creates a new inspection.
2. Inspection record starts as ``in_progress``.
3. Inspector updates details such as mileage, fuel level, notes, checklist data, accessories, and optional GPS coordinates.
4. Damage records are added via a vehicle diagram on top, front, rear, left, and right views.
5. Photos are uploaded and attached to the inspection or to specific damage records.
6. Inspector and optionally customer signatures are saved.
7. Submit endpoint validates:
 - mileage is present
 - fuel level is present
 - at least one photo exists
8. Inspection status becomes ``submitted``.
9. Privileged users can later review, approve, dispute, or add review notes through update routes.

9.3 Mobile UX Support

The inspection component set includes:

- ``MobileInspectionForm`` multi-step wrapper
- ``PhotoCapture`` with ``capture="environment"`` for camera-first uploads
- ``SignatureCanvas`` for touch signatures
- ``VehicleDiagram`` with tappable damage pin placement
- ``StarRating`` and other form widgets

9.4 Stored Inspection Artifacts

- inspection master record
- damage records
- photo metadata rows
- physical photo files
- signature image files

10. API Surface

This section documents the routes present in ``src/app/api``.

10.1 Auth APIs

``POST /api/auth/login``

- validates password against ``users.password_hash``
- updates ``last_login_at``

- sets JWT cookie
- GET /api/auth/me`
- resolves authenticated user profile and branch data
- PUT /api/auth/change-password`
- validates current password
 - writes new password hash
 - clears `must\_change\_password`
- GET /api/auth/sso`
- redirects to Microsoft authorization endpoint
- GET /api/auth/sso/callback`
- exchanges code for tokens
 - resolves user profile through Microsoft Graph or ID token
 - matches local user by normalized email
 - updates last login
 - issues JWT cookie
- POST /api/auth/logout`
- logs logout and clears cookie

10.2 Fleet APIs

- GET /api/fleet`
- returns filtered/paged vehicles plus fleet stats
- GET /api/fleet/stats`
- returns summary stats used by dashboard
- GET /api/fleet/[assetnum]`
- returns vehicle detail merged with platform override data
- PUT /api/fleet/[assetnum]/overrides`
- role-protected write endpoint
 - validates override values
 - verifies asset exists in Maximo
 - upserts platform override record

10.3 Booking APIs

- GET /api/bookings`
- supports status and month filters plus pagination
- POST /api/bookings`
- creates booking with UUID
 - stores creator user ID when available

`GET /api/bookings/calendar`

- returns bookings overlapping a given month

`PUT /api/bookings/[id]`

- implied by UI usage for status updates and cancellation

`DELETE /api/bookings/[id]`

- implied by UI usage for permanent removal

Note:

The `[id]` booking route exists in the repo even though it was not expanded in this documentation capture. The page code clearly depends on it for update and delete operations.

10.4 Customer APIs

`GET /api/customers`

- reads from Maximo `PLUSPCUSTOMER`
- supports search
- supports active-rentals-only mode by joining `ASSET`

10.5 Inspection APIs

`GET /api/inspections`

- list with type, status, vehicle filters and paging

`POST /api/inspections`

- creates inspection linked to current user

`GET /api/inspections/[id]`

- returns inspection plus damages and photos

`PUT /api/inspections/[id]`

- updates many inspection fields
- ownership and privilege checks apply

`POST /api/inspections/[id]/submit`

- validates required fields and photo existence
- transitions to `submitted`

`GET|POST /api/inspections/[id]/damages`

- list or create damage records

`PUT|DELETE /api/inspections/[id]/damages/[damageId]`

- update or delete specific damage records

`GET|POST|DELETE /api/inspections/[id]/photos`

- list photos
- upload photo using multipart form data
- delete uploaded photo and backing file

`POST /api/inspections/[id]/signature`

- saves inspector or customer signature PNG

10.6 Service APIs

`GET /api/services`

- paged list of Maximo work orders with filters

`GET /api/services/[wonum]`

- work-order detail with labor lines and material lines

10.7 Analytics APIs

`GET /api/analytics`

- combines AX / D365 invoice and agreement data with Maximo work-order and fleet status distributions
- derives top customers, top orders, and revenue segmentation

10.8 AI APIs

`POST /api/ai/chat`

- streams OpenAI responses over SSE
- injects live fleet context from Maximo

`GET /api/ai/forecast`

- returns time-series forecast data using exponential smoothing

`GET /api/ai/maintenance`

- computes maintenance priority scores from cost, age, service recency, and open work orders

`GET /api/ai/anomalies`

- flags cost spikes, high repair frequency, and long-idle vehicles

10.9 User APIs

`GET /api/users`

- super-admin-only listing with filters and paging

`POST /api/users`

- creates new user
- default temporary password is `Temp@123` when not supplied
- marks `must\_change\_password = 1`

`GET /api/users/[id]`

- fetches a single user

`PUT /api/users/[id]`

- updates user data
- can also reset password to `Temp@123`

`PUT /api/users/[id]/deactivate`

- prevents self-deactivation
- marks account `inactive`

10.10 File API

`GET /api/files/[...path]`

- authenticated file retrieval
- serves inspection photos, signatures, and any stored files based on path

11. Data Model Summary

The repository directly references these key platform-owned tables:

- `users`
- `branches`
- `vehicle\_categories`
- `vehicle\_overrides`
- `bookings`
- `inspections`
- `damage\_records`
- `inspection\_photos`
- `audit\_logs`

Enterprise system tables referenced in query logic include:

- Maximo:
- `asset`
- `workorder`
- `labtrans`
- `matusetrans`
- `PLUSPCUSTOMER`
- AX / D365:
- `SALESTABLE`
- `CUSTINVOICEJOUR`

12. PWA And Mobile Support

12.1 Manifest

`public/manifest.json` defines:

- standalone display
- portrait orientation

- app icons
- brand colors

12.2 Service Worker

``public/sw.js``:

- caches offline page and logo
- serves offline fallback for failed navigation requests
- uses cache name ``gbcv-v1``

12.3 Registration

``ServiceWorkerRegistration.tsx`` registers the service worker on the client if supported.

12.4 Mobile-Specific UX Patterns

Observed throughout the app:

- mobile headers and bottom nav
- responsive tables that collapse to cards
- floating action button for creation flows
- touch-friendly inspection tools
- bottom-sheet / responsive modal infrastructure

13. Real-Time Event Layer

``server.js`` runs a Socket.io server on port ``3002`` by default.

Defined event channels:

- ``fleet:updated``
- ``fleet:statusChange``
- ``booking:created``
- ``booking:updated``
- ``booking:cancelled``
- ``ai:forecast``
- ``ai:anomaly``

The main dashboard pages currently rely mostly on polling and fetch calls, so the event layer exists as supporting infrastructure rather than a universally wired experience.

14. Environment Variables

From ``env.example``, expected runtime configuration includes:

14.1 Maximo

- ``MAXDB_SERVER``
- ``MAXDB_DATABASE``
- ``MAXDB_USER``
- ``MAXDB_PASSWORD``
- ``MAXDB_PORT``

14.2 Platform Database

- ``PLATFORM_SERVER``
- ``PLATFORM_DATABASE``
- ``PLATFORM_USER``
- ``PLATFORM_PASSWORD``
- ``PLATFORM_PORT``

14.3 Write-Capable App Database

- ``DB_SERVER``
- ``DB_NAME``
- ``DB_USER``
- ``DB_PASSWORD``
- ``DB_PORT``

14.4 Auth And Security

- ``JWT_SECRET``
- ``AZURE_AD_TENANT_ID``
- ``AZURE_AD_CLIENT_ID``
- ``AZURE_AD_CLIENT_SECRET``
- ``AZURE_AD_REDIRECT_URI``
- ``USE_HTTPS`` if secure cookie behavior is needed in production

14.5 Files, AI, And Socket

- ``FILE_STORAGE_ROOT``
- ``OPENAI_API_KEY``
- ``SOCKET_PORT``

14.6 AX / D365

Required by ``axdb.ts``:

- ``AXDB_SERVER``
- ``AXDB_DATABASE``
- ``AXDB_USER``

- ``AXDB_PASSWORD``
- ``AXDB_PORT``
- optional TLS toggles:
- ``DB_ENCRYPT``
- ``DB_TRUST_SERVER_CERT``

15. Local Development And Startup

The codebase exposes these package scripts:

- ``npm run dev``
- ``npm run build``
- ``npm run start``
- ``npm run socket``
- ``npm run lint``

Typical local startup:

1. populate ``.env.local``
2. run ``npm install``
3. run ``npm run dev``
4. optionally run ``npm run socket`` in a second terminal

The dev server binds ``0.0.0.0``, making device testing easier on local networks.

16. Existing Design Documents In The Repo

The `docs/superpowers/specs`` and `docs/superpowers/plans`` directories show prior structured planning for:

- foundation architecture
- vehicle fleet enhancements
- mobile app architecture
- mobile-responsive/PWA adaptation

These are useful references when deciding whether current behavior is:

- fully implemented
- partially implemented
- still planned only

The current repository most visibly implements:

- core dashboard shell
- fleet management
- booking management

- user management
- inspection foundation and mobile-first mechanics
- responsive/PWA support

17. Observed Gaps, Risks, And Technical Debt

This is a documentation section, not a formal code review, but several implementation realities are important for future maintainers:

- `README.md` is still the default Next.js template and does not describe the actual product.
- Some routes implied by UI navigation were not observed in the app routes, for example customer detail.
- Several pages are large client components with embedded fetch logic and local state rather than shared data hooks or server components.
- SQL queries are written inline in route handlers, which makes behavior explicit but increases maintenance cost.
- The system depends heavily on live enterprise databases being reachable.
- The AI features currently use direct OpenAI calls and basic statistical heuristics rather than a dedicated AI service boundary.
- Socket.io infrastructure exists, but large parts of the UI still use polling or one-shot fetches.
- The working tree currently contains uncommitted modifications in dashboard and shared UI files, so runtime behavior may differ slightly from the last git commit.
- Some DB helpers include fallback credentials or defaults directly in code. This should be treated carefully in any security hardening effort.

18. Recommended Maintenance Priorities

For future project continuity, the highest-value follow-on documentation and engineering tasks would be:

1. replace the generic README with a project-specific setup and architecture overview
2. document the database schemas formally
3. document any missing routes or planned-but-unimplemented screens
4. centralize API contracts and response envelopes
5. separate enterprise query logic from route handlers into service modules
6. add operational deployment documentation
7. define a roadmap for booking/dispatch optimization if that remains the main business priority

19. Reference Entry Points

These files are the fastest way for a new maintainer to orient themselves:

- `src/app/(dashboard)/layout.tsx`
- `src/app/(dashboard)/page.tsx`

- ``src/app/api/auth/login/route.ts``
- ``src/app/api/fleet/route.ts``
- ``src/app/api/bookings/route.ts``
- ``src/app/api/inspections/route.ts``
- ``src/app/api/analytics/route.ts``
- ``src/app/api/users/route.ts``
- ``src/lib/auth.ts``
- ``src/lib/db.ts``
- ``src/lib/maxdb.ts``
- ``src/lib/axdb.ts``
- ``src/lib/file-storage.ts``

20. Conclusion

The GBCR Platform repository already represents a substantial operational product rather than a starter app. Its strongest implemented areas are dashboard shell, fleet visibility, booking operations, inspections, services lookup, analytics, and role-based admin features. Its architecture is practical and integration-heavy, with the application serving as an orchestration layer over platform-owned tables and external enterprise systems.

For reference use, the most important mental model is this:

- Next.js provides the shell and API surface.
- ``GBCR_Platform`` stores application-owned workflows and permissions.
- Maximo provides live fleet and maintenance truth.
- AX / D365 provides financial and agreement analytics.
- Microsoft Entra and JWT cookies gate access.
- inspection assets are stored in file storage and referenced by DB rows.

That model explains most of the codebase.